Algorithmes et Pensée Computationnelle

Algorithmes de tri et Complexité - exercices avancés

Le but de cette série d'exercices est d'aborder les notions présentées durant la séance de cours. Cette série d'exercices sera orientée autour des points suivants :

1. La récursivité
2. La complexité des algorithmes
3. Les algorithmes de tri

Les langages de programmation qui seront utilisés pour cette série d'exercices sont Java et Python.
Le temps indiqué (🕓) est à titre indicatif.

1 Récursivité (20 minutes)

Question 1: (🕓 *5 minutes*) **Fibonacci**
La suite de Fibonacci est définie récursivement par les propriétés suivantes :

— si n est égal à 0 ou 1 : fibo(0) = fibo(1) = 1

— si n est supérieur ou égal à 2, alors ; fibo (n) = fibo(n - 1) + fibo(n - 2)

Voici son implémentation en Java :

```java
public static int fibonacci(int n) {
    if(n == 0 || n == 1){
        return n;
    } else{
        return fibonacci(n-1) + fibonacci(n-2);
    }
}
```

Quel est la complexité de l'algorithme ci-dessus ?

Conseil

Pour formaliser la formule de complexité, soit $T(n)$ le nombre d'opérations requises pour calculer `fibonacci(n)`. Ainsi, $T(n) = T(n-1) + T(n-2) + c$, c étant une constante et $n \geq 2$. Dans cette formule, $T(n-1)$ est le nombre d'opérations de `fibonacci(n-1)` et $T(n-2)$ est le nombre d'opérations de `fibonacci(n-2)`. À l'aide de cette formule récursive et du fait que $T(0)$ et $T(1)$ valent 1, déduisez les premières valeurs de $T(n)$ et tracez-les.

1. $O(n^2)$
2. $O(n)$
3. $O(\log(n))$
4. $O(2^n)$

Solution

La complexité de cet algorithme est $O(2^n)$.

Preuve : $F(n) \geq 2^{n/2}$ pour tout $n \geq 6$. Ceci établit une croissance exponentielle de base $\sqrt{2} \approx 1.414$.
Cas de base :

— $F(6) = 8 \geq 2^3 = 8$

— $F(7) = 13 \geq 2^{3.5} \approx 11.3$

Solution

Étape de récurrence : Supposons que la borne soit vraie pour tout $k \leq n$. Alors :

$$F(n+1) = F(n) + F(n-1) \geq 2^{n/2} + 2^{(n-1)/2} = 2^{(n-1)/2}(\sqrt{2}+1)$$

Puisque $\sqrt{2}+1 \approx 2.414 > 2 = (\sqrt{2})^2$, nous avons :

$$F(n+1) \geq 2^{(n-1)/2} \cdot (\sqrt{2})^2 = 2^{(n-1)/2} \cdot 2^1 = 2^{(n+1)/2}$$

Ceci achève la récurrence.

Question 2: (*15 minutes*) **- Fibonacci et utilisation de la mémoire - Python**

Le calcul des éléments de la suite de Fibonacci en utilisant l'algorithme de la question 1 est fastidieux. Cela est dû au fait que certaines opérations sont effectuées plusieurs fois. Vous pouvez le vérifier en passant un grand nombre à la fonction `fibonacci`. Par exemple `fibonacci(50)` peut prendre plusieurs heures avant de donner un résultat !
En Python, simplifiez cet algorithme en écrivant une fonction `fibonacci` qui prend en entrée un nombre `n` et un dictionnaire `previous_fibonacci`. Le dictionnaire stockera chacune des opérations des appels récursifs. Ce dictionnaire aura pour clé un nombre `n` et pour valeur le résultat de l'appel de la fonction `fibonacci(n)`.
Avant de faire appel à la fonction de façon récursive, vérifiez que le nombre passé en paramètre n'existe pas dans le dictionnaire `previous_fibonacci`.

Conseil

Au lancement du programme, `previous_fibonacci` aura pour valeur `{0:0, 1:1}` ce qui correspond aux conditions de sortie des appels récursifs.
La fonction `fibonacci` retournera le dernier élément du dictionnaire `previous_fibonacci`.

Solution

Sans utilisation de la mémoire l'algorithme avait une complexité de $O(2^n)$, maintenant de $O(n)$.

```
1  counter = 0
2  previous_fibonacci = {0:0, 1:1}
3
4  def fibonacci(number, previous):
5      global counter
6      counter += 1
7      if number not in previous:
8          previous[number] = fibonacci(number - 1, previous) +
       fibonacci(number - 2, previous)
9
10     return previous[number]
11
12 if __name__ == "__main__":
13     print(fibonacci(50, previous=previous_fibonacci))
```

2 Algorithmes de Tri (40 minutes)

Question 3: (*20 minutes*) **Tri à bulles (Bubble Sort) - Java** Optionnel
Le tri à bulles consiste à parcourir une liste et à comparer ses éléments. Le tri est effectué en permutant les éléments de telle sorte que les éléments les plus grands soient placés à la fin de la liste.

Concrètement, si un premier nombre x est plus grand qu'un deuxième nombre y et que l'on souhaite trier l'ensemble par ordre croissant, alors x et y sont mal placés et il faut les inverser. Si, au contraire, x est plus petit que y, alors on ne fait rien et l'on compare y à z, l'élément suivant.
Soit la liste `l = [1, 2, 4, 3, 1]`, triez les éléments de la liste en utilisant un tri à bulles. Combien d'itérations effectuez-vous ?

— **Java :**

```java
public class BubbleSort {
    public static void triBulle(int[] l) {
        for (int i = 0; i < l.length - 1; i++) {
            // TODO: Code à compléter
        }
    }

    public static void printArray(int l[]) {
        int n = l.length;
        for (int i = 0; i < n; ++i) {
            System.out.print(l[i] + " ");
        }
        System.out.println();
    }

    public static void main(String[] args) {
        int[] l = { 1, 2, 4, 3, 1 };
        BubbleSort.triBulle(l);
        printArray(l);
    }
}
```

Conseil

En Java, utilisez une variable temporaire que vous nommerez `temp` afin de faire l'échange de valeur entre deux éléments de la liste.

Solution

Java :

```java
public class BubbleSort {
    public static void triBulle(int[] l) {
        int n = l.length;
        for (int i = 0; i < n - 1; i++){
            for (int j = 0; j < n-i-1; j++) {
                if (l[j] > l[j+1]) {
                    // échange l[j+1] et l[i]
                    int temp = l[j];
                    l[j] = l[j+1];
                    l[j+1] = temp;
                }
            }
        }
    }
```

Solution

```java
    public static void printArray(int l[]) {
        int n = l.length;
```

```java
        for (int i = 0; i < n; ++i)
            System.out.print(l[i] + " ");

        System.out.println();
    }

    public static void main(String[] args) {
        int[] l = { 1, 2, 4, 3, 1 };
        triBulle(l);
        printArray(l);
    }
}
```

Solution

L'algorithme a une complexité de $O(n^2)$ car il contient deux boucles imbriquées qui parcourent la liste.

Question 4: (*20 minutes*) **Tri par insertion (Insertion Sort)**

Dans l'algorithme de tri par insertion, on parcourt le tableau à trier du début à la fin. Au moment où on considère le i-ème élément, les éléments qui le précèdent sont déjà triés. Pour faire l'analogie avec l'exemple du jeu de cartes, lorsqu'on est à la i-ème étape du parcours, le i-ème élément est la carte saisie, les éléments précédents sont la main triée et les éléments suivants correspondent aux cartes encore en désordre sur la table.
L'objectif d'une étape est d'insérer le i-ème élément à sa place parmi ceux qui le précède. Il faut pour cela trouver où l'élément doit être inséré en le comparant aux autres, puis décaler les éléments afin de pouvoir effectuer l'insertion. En pratique, ces deux actions sont fréquemment effectuées en une passe, qui consiste à faire "remonter" l'élément au fur et à mesure jusqu'à rencontrer un élément plus petit.
Compléter le code suivant pour trier la liste `l` définie ci-dessous en utilisant un tri par insertion. Combien d'itérations effectuez-vous ?

— **Python :**

```python
def tri_insertion(l):
    for i in range(1, len(l)):
        #TODO: Code à compléter

if __name__ == "__main__":
    l = [2, 43, 1, 3, 43]
    tri_insertion(l)
    print(l)
```

— **Java :**

```java
public class InsertionSort {
    public static void triInsertion(int[] l) {
        for (int i = 1; i < l.length; i++) {
            // TODO: Code à compléter
        }
    }

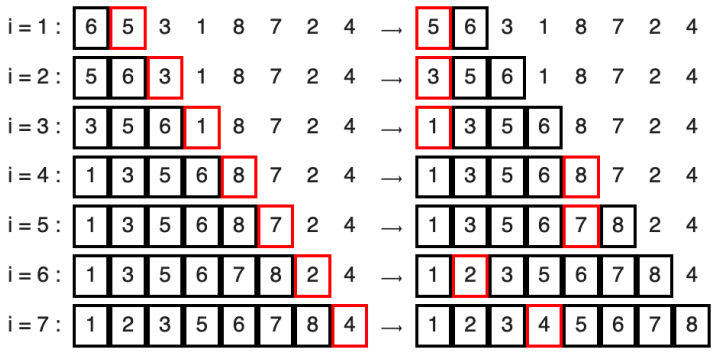
    public static void printArray(int l[]) {
        int n = l.length;
        for (int i = 0; i < n; ++i) {
            System.out.print(l[i] + " ");
        }
    }

    public static void main(String[] args) {
        int[] l = { 2, 43, 1, 3, 43 };
```

```
17            InsertionSort.triInsertion(l);
18            printArray(l);
19        }
20    }
```

Conseil

La figure ci-dessous illustre un exemple de tri par insertion.
Référez-vous aussi aux diapositives 15 à 70 du cours.



Solution

Python :

```python
def tri_insertion(l):
    for i in range(1, len(l)):
        key = l[i]
        j = i - 1

        while j >= 0 and key < l[j]:
            l[j + 1] = l[j]
            j -= 1
        l[j + 1] = key


if __name__ == "__main__":
    l = [2, 43, 1, 3, 43]
    tri_insertion(l)
    print(l)
```

Solution

Java :

```java
public class InsertionSort {
    public static void triInsertion(int[] l) {
        for (int i = 1; i < l.length; i++){
            int key = l[i];
            int j = i - 1;

            while (j >= 0 && l[j] > key) {
                l[j + 1] = l[j];
                j = j - 1;
            }
            l[j + 1] = key;
        }
```

```java
    }

    public static void printArray(int l[]){
        int n = l.length;
        for (int i = 0; i < n; ++i)
            System.out.print(l[i] + " ");
    }

    public static void main(String[] args){
        int[] l = {2, 43, 1, 3, 43};
        triInsertion(l);
        printArray(l);
    }
}
```

Solution

La complexité de l'algorithme est de $O(n^2)$ car nous utilisons 2 boucles imbriquées. Dans le pire des cas, la boucle intérieure parcourt la liste une fois pour chaque tour de la boucle extérieure.

Question 5: (*20 minutes*) **Tri fusion (Merge Sort) - Java**

À partir de deux listes triées, on peut facilement construire une liste triée comportant les éléments issus de ces deux listes (leur *fusion*). Le principe de l'algorithme de tri fusion repose sur cette observation : le plus petit élément de la liste à construire est soit le plus petit élément de la première liste, soit le plus petit élément de la deuxième liste. Ainsi, on peut construire la liste élément par élément en retirant tantôt le premier élément de la première liste, tantôt le premier élément de la deuxième liste (en fait, le plus petit des deux, à supposer qu'aucune des deux listes ne soit vide, sinon la réponse est immédiate).
Les étapes à suivre pour implémenter l'algorithme sont les suivantes :

1. Si le tableau n'a qu'un élément, il est déjà trié.
2. Sinon, séparer le tableau en deux parties plus ou moins égales.
3. Trier récursivement les deux parties avec l'algorithme de tri fusion.
4. Fusionner les deux tableaux triés en un seul tableau trié.

Soit la liste `l` suivante [38, 27, 43, 3, 9, 82, 10], triez les éléments de la liste en utilisant un tri fusion. Combien d'itération effectuez-vous ?

— **Java :**

```java
public class MergeSort {
    // Fusionne 2 sous-listes de arr[].
    // Première sous-liste est arr[l..m]
    // Deuxième sous-liste est arr[m+1..r]
    public static void merge(int arr[], int l, int m, int r) {
        // TODO: Code à compléter
    }

    // Fonction principale qui trie arr[l..r] en utilisant
    // merge()
    public static void triFusion(int arr[], int l, int r) {
        // TODO: Code à compléter
    }

    public static void printArray(int l[]) {
        int n = l.length;
        for (int i = 0; i < n; ++i) {
            System.out.print(l[i] + " ");
        }
        System.out.println();
    }
```

```
22
23      public static void main(String[] args) {
24          int[] l = { 38, 27, 43, 3, 9, 82, 10 };
25          MergeSort.triFusion(l, 0, l.length - 1);
26          printArray(l);
27      }
28  }
```

Conseil

— L'algorithme est récursif.

— Revenez à la visualisation de l'algorithme dans les diapositives du cours pour comprendre comment marche concrètement le tri fusion.

Solution

Java :

```
1   public class MergeSort {
2       // Fusionne 2 sous-listes de arr[].
3       // Première sous-liste est arr[l..m]
4       // Deuxième sous-liste est arr[m+1..r]
5       public static void merge(int arr[], int l, int m, int r) {
6           // Trouver la taille des deux sous-listes à fusionner
7           int n1 = m - l + 1;
8           int n2 = r - m;
9
10          /* Créer des listes temporaires */
11          int leftArray[] = new int[n1];
12          int rightArray[] = new int[n2];
13
14          /* Copier les données dans les sous-listes temporaires */
15          for (int i = 0; i < n1; ++i) {
16              leftArray[i] = arr[l + i];
17          }
18          for (int j = 0; j < n2; ++j) {
19              rightArray[j] = arr[m + 1 + j];
20          }
```

Solution

```
21
22          /* Fusionner les sous-listes temporaires */
23          // Indexes initiaux de la première et seconde sous-liste
24          int i = 0, j = 0;
25
26          // Index initial de la sous-liste fusionnée
27          int k = l;
28          while (i < n1 && j < n2) {
29              if (leftArray[i] <= rightArray[j]) {
30                  arr[k] = leftArray[i];
31                  i++;
32              } else {
33                  arr[k] = rightArray[j];
34                  j++;
35              }
36              k++;
37          }
38
39          /* Copier les élements restants de leftArray[] */
```

```
40          while (i < n1) {
41              arr[k] = leftArray[i];
42              i++;
43              k++;
44          }
45
46          /* Copier les élements restants de rightArray[] */
47          while (j < n2) {
48              arr[k] = rightArray[j];
49              j++;
50              k++;
51          }
52      }
53
54      // Fonction principale qui trie arr[l..r] en utilisant
55      // merge()
56      public static void triFusion(int arr[], int l, int r) {
57          if (l < r) {
58              // Trouver le milieu de la liste
59              int m = (l + r) / 2;
60
61              // Trier les première et la deuxième parties de la liste
62              triFusion(arr, l, m);
63              triFusion(arr, m + 1, r);
64
65              // Fusionner les deux parties
66              merge(arr, l, m, r);
67          }
68      }
69
70      public static void afficheListe(int l[]) {
71          int n = l.length;
72          for (int i = 0; i < n; ++i) {
73              System.out.println(l[i] + " ");
74          }
75      }
76
77      public static void main(String[] args) {
78          int[] l = { 38, 27, 43, 3, 9, 82, 10 };
79          triFusion(l, 0, l.length - 1);
80          afficheListe(l);
81      }
82  }
```

Solution

Le tri fusion est un algorithme récursif. Ainsi, nous pouvons exprimer sa complexité temporelle via une relation de récurrence : $T(n) = 2T(n/2) + O(n)$. En effet, l'algorithme comporte 3 étapes :

1. "Divide Step", qui divise les listes en deux sous-listes, et cela prend un temps constant
2. "Conquer Step", qui trie récursivement les sous-listes de taille $n/2$ chacune, et cette étape est représentée par le terme $2T(n/2)$ dans l'équation.
3. La dernière étape consiste à fusionner les listes, sa complexité est de $O(n)$.

La solution à cette équation est $O(n \log n)$.